Von: **Arvind and Sayash from AI as Normal Technology** aisnakeoil@substack.com
Betreff: Why AI hasn't replaced software engineers, and won't
Datum: 11. Juni 2026 um 04:30
An: johannes.meier@xigmbh.de

AI AS NORMAL TECHNOLOGY

Why AI hasn't replaced software engineers, and won't

Coding agents as normal technology

ARVIND NARAYANAN AND SAYASH KAPOOR

JUN 11

There is great anxiety and uncertainty about AI replacing jobs. How can we move past vague warnings and bombastic predictions and bring data to bear on this question? One good way is to look at the profession where AI capabilities are furthest along and adoption has been exceptionally rapid: software engineering.

In this essay, we argue that there is enough evidence to reject the narrative that once AI capabilities reach a certain threshold, it will cause mass layoffs. Given that this is true even in a sector with very few regulatory barriers, most other professions are likely to be even more cushioned.

We also have a good understanding of why this is the case. We can think of many kinds of knowledge work, including software development, as a "decide-execute-deliver sandwich". AI compresses the "execute" layer — the

middle of the sandwich — but the other two layers resist automation in a way that will not be overcome by capability improvements alone.

We conclude on a note of cautious optimism about the future trajectory of demand for software engineering. This essay is the first in a series, and the next one will look at reasons why individual software engineers' careers might be rocky even if overall demand is healthy. The series is based on the published literature in economics and software engineering, our own evaluations and observations of AI agents, and many software engineers' reflection on the present and future of AI impacts on their profession, gleaned both from published writings and our interactions with the community.

The stories of AI-driven mass layoffs in software seem to be classic "AI washing"

Consider three stories that made the headlines and how they contrasted with reality:

- In February, fintech company Block (maker of Cash App, Square, Afterpay, and other such apps) announced layoffs of 4,000 employees because, according to founder Jack Dorsey, AI is "enabling a new way of working" with "smaller and flatter teams", specifically citing late-2025 improvements in model capabilities.

  But subsequent reporting revealed a radically different picture. After growing headcount more than threefold during the pandemic, the company was under massive financial pressure. A data scientist on the Cash App team, Naoko Takeda posted that Block "shoved AI down everyone's throats" yet she saw "very limited gains in productivity." She refused a 75% retention raise and quit. Other employees interviewed had a sharply different understanding of what AI was capable of at Block and whether Dorsey had a competent understanding of the issues.

  As Aaron Levie has pointed out, CEOs are uniquely prone to delusions about AI's usefulness because they can build quick prototypes but can't see the 90% of work it takes to turn it into a finished product. Dorsey's

public statements about AI seem to fit exactly this pattern.

- In April, Snap laid off about 1,000 people, with CEO Evan Spiegel primarily citing AI as the reason in his layoff memo. He also said that AI generated 65% of new code. In reality, the layoffs followed a campaign by an activist investor demanding cost cuts. (Snap has posted a net loss every full year since its 2017 IPO and shares were down over 30% in 2026). Tellingly, the nature of the cuts, such as 150 jobs spanning various roles in the augmented reality division, don't correlate with the cuts we would expect to see if they were driven by AI (i.e. programming and other "AI-exposed" jobs across the board, not concentrated in any unit).
- In May, Intuit announced 3,000 cuts, alongside deals with Anthropic and OpenAI. The press connected the two, framing the layoffs as AI-driven restructuring. For once, the CEO actually pushed back on this easy narrative, saying that "none of it had to do with AI" and that the cuts targeted "coordination-heavy roles" and too many management layers.

We did not cherry-pick these examples. In every story about AI-driven software engineering layoffs that we examined, the same narrative violation emerged. It turns out that "AI washing" of job cuts is an economy-wide phenomenon, evidenced by many surveys:

- 59% of U.S. hiring managers admitted they emphasize AI when explaining hiring freezes or layoffs because it plays better with stakeholders than citing financial constraints.
- Forrester principal analyst J. P. Gownder says of companies preparing supposedly AI-driven layoffs: "When we ask if they have a mature, vetted AI app ready to fill in those jobs, nine out of 10 times, the answer is no—and they haven't even started."
- In a HBR survey of over 1,000 global executives, 21% had made large headcount reductions "in anticipation of" AI, with another 39% having made low or moderate anticipatory headcount reductions. In contrast, only 2% had already made large reductions in headcount related to actual AI implementation. The 10x gap suggests that executives, like everyone else, are highly prone to succumbing to the misleading

narratives about AI replacing jobs.

Another interesting data point comes from the WARN Act, which requires certain disclosures of plant closings and mass layoffs affecting over 100 workers. In March 2025, New York became the first U.S. state to add an AI disclosure checkbox to WARN Act filings. In the full first year, more than 160 companies filed WARN notices. Not a single one checked the AI box.[1] We reached out to the NY Department of Labor who confirmed that as of late May, only one company, Nespresso, checked the box.[2] If these filings are accurate, only 46 out of about 25,000 laid off workers in New York State in the relevant period, or about two-tenths of a percent, were affected by AI.

Even more damning for the AI-driven-mass-layoffs narrative: layoffs are the wrong signal of AI's potential productivity benefits in the first place! The research is clear that the effect operates through "slower hiring rather than increased separations". Firing existing workers results in the loss of precisely the tacit knowledge and organizational capital that allows workers to operate AI effectively. Besides, it is expensive in terms of severance, damage to morale, and rehiring risk. Given these costs, it is largely unnecessary given that natural turnover achieves the same result in a few years.

So what does the data tell us when we look beyond layoffs to overall employment trends? An important paper from Federal Reserve economists compiles the evidence in the U.S. context. Employment is still *growing*, but they find that it is growing slower post-ChatGPT compared to a no-AI counterfactual, by about 3 percentage points per year. One important limitation of this study is that the methodology can't capture self-employment, so it is possible that some of the slowdown in growth is being absorbed by entrepreneurship instead. We do have evidence from other studies that AI makes entrepreneurship easier. So the real picture is probably even healthier than the Federal Reserve study suggests.[3]

Finally, it is worth acknowledging two kinds of indirectly-AI-driven job losses in software engineering that are real, but different from AI replacing software engineers. First, AI sometimes decimates demand for the product, in cases like Chegg (homework help) or Stack Overflow (technical help), both of which

have laid off workers. AI doesn't directly do the job that these workers did, but rather obviates the need for it. The historical parallel is strong: Among the 270 jobs in the 1950 U.S. census, only one job was automated away — elevator operator. But many others were rendered obsolete by new technology, like the job of telegraph operator.

Another credible AI-driven layoffs story is among companies that *sell* AI, rather than *buy* it. So when companies like IBM or SAP announce layoffs because of AI, a more accurate framing is "we reallocated headcount from legacy functions to our fastest-growing product line." That's ordinary corporate restructuring around a revenue opportunity, not technology displacing workers.

Why coding agents haven't led to labor displacement: the decide-execute-deliver sandwich

Many tech leaders, like the Snap CEO above, report the percentage of code written by AI alongside reports of layoffs or predictions of future job losses. This feeds into the simplistic mental model that once AI writes all the code, there is no need for coders. Fortunately, this mental model is wrong. This AI-written-code metric is almost completely disconnected from what matters for labor displacement. Here's why.

First, writing code isn't, and never was, the bottleneck. For example, a 2019 paper summarized existing studies with the conclusion that "developers spend surprisingly little time with coding, 9% to 61% depending on the study". This finding was consistent with the paper's own data from 6,000 developers at Microsoft. As coding agents began to be taken up, there was an explosion of blog posts in late 2025 pointing out that writing code isn't the bottleneck, as developers realized that using agents to write most of the code led to little impact on overall productivity [1, 2, 3, 4, 5, 6, 7, 8].

If writing code isn't the bottleneck, what is? The task-breakdown surveys point at things like meetings or debugging. This just leads to more questions: what are developers doing in those meetings and why can't it be done by AI? Won't debugging get automated as capabilities improve? To understand the

real bottlenecks, we have to get qualitative, and dig into software engineers' own understanding of what it is they do that resists automation.

When we did this analysis, it revealed three things as the real bottlenecks (1) deciding and specifying what to build, (2) verifying and being accountable for what is delivered, and (3) the deep human understanding — of the codebase, the business, and the environment — required to carry out both of these.

In other words, software engineers' work consists of a "decide-execute-deliver" sandwich (with understanding being a prerequisite for all three). AI has compressed the middle of the sandwich, but has left the two ends largely unchanged. As long as software development teams are in charge of decision making and accountable for what they deliver, engineers still need to spend time building up a deep understanding of the system. These are the three bottlenecks.

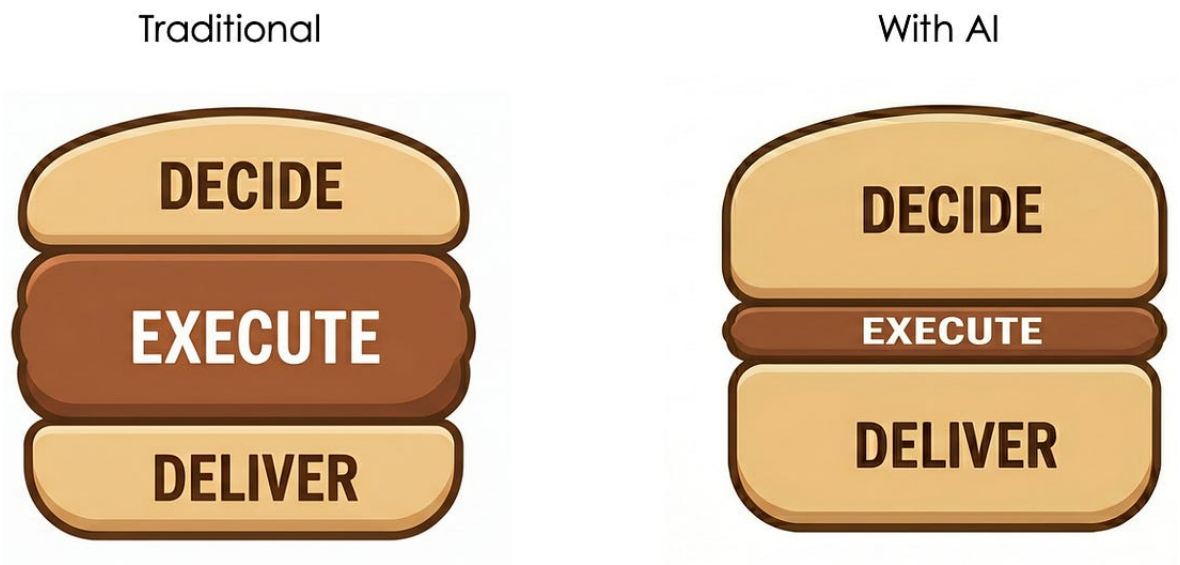


Figure: Software development consists of three layers: (1) Decision making — problem framing, specification, planning (2) execution — design and implementation (3) delivery — testing, verification, integration, maintenance, etc. Note that these are conceptual layers, not temporal phases. It is common to switch back and forth in the course of a project.

Evidence for the sandwich model of AI's productivity effects comes from a

recent paper on "Writing Code vs. Shipping Code". Across 100,000 developers on GitHub, the researchers found that AI agents led to an eight-fold increase in the number of lines of code written, consistent with the idea that AI almost completely compresses the Execute layer of the sandwich. But this led to only 30% more releases, strongly suggesting that human bottlenecks (the Decide and Deliver layers) remain in place.[4]

Can the sandwich be further compressed? We don't think so. At one end of the pipeline, development teams need to decide what to build. One of the most important lessons junior software engineers learn is that requirements specification (the profession's lingo for this layer) takes surprisingly long, and if it is compressed, it leads to much more pain down the line. This layer is hard to automate because it requires thinking about user needs, market signals, organizational priorities, and in some cases regulatory constraints.

As AI capabilities improve, the kinds of decisions that can be delegated to AI increase over time. But this does not make the "decide" layer thinner — once a decision can be delegated to AI, it is no longer a source of competitive advantage, and the value of human decision-making migrates upward. Software increases in complexity over time, so there is no ceiling to this process.

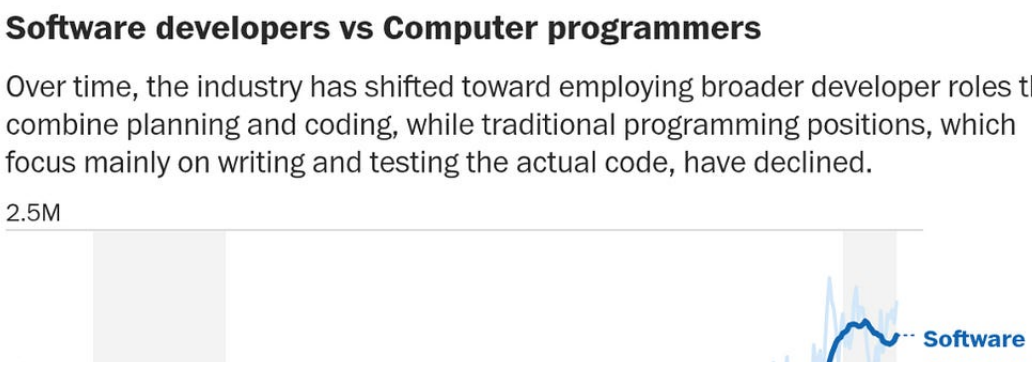
At the other end of the sandwich, human teams need to be accountable for what they deliver. It is possible that some day in the future teams will ship mission-critical code without fully testing and understanding it, but today's AI is so unreliable that such haphazard practices would represent an existential threat to software teams and their customers.

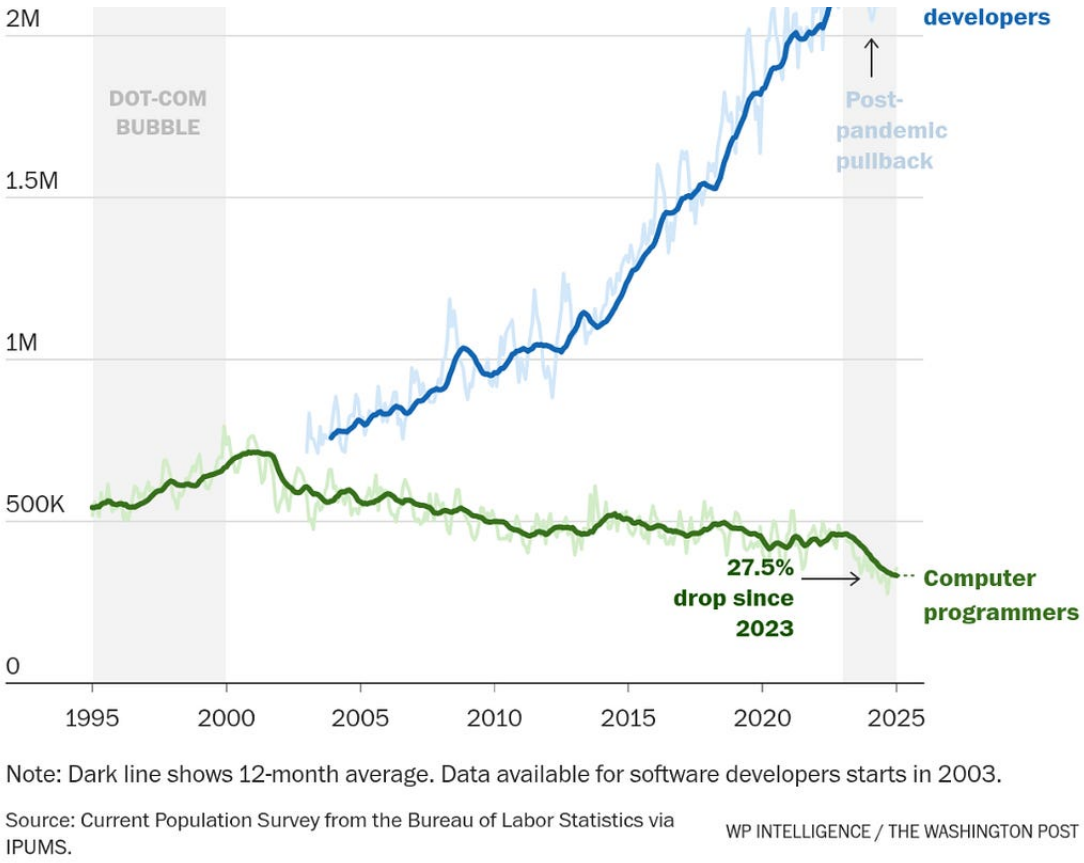
Even if the technical barriers go away in the future, we don't have to cede control to AI. A central insight of *AI as Normal Technology* is that we can collectively choose to keep humans accountable through shared norms, law, and policy. This is a much more resilient way to control the speed of AI impacts and improve safety than trying to slow the development of technical capabilities. These speed barriers are already largely in place due to liability laws and sector-specific regulation, but can be further strengthened. (For a longer version of this argument, see the original essay.)

In this vision, as more and more of the execution layer gets delegated to AI, the software engineer's role in the future becomes analogous to that of a crane operator. AI agents will do most of the cognitive heavy lifting; supervising the agent and keeping it in control becomes most of the human's job.

Some commentators argue that a future with humans staying in control is unlikely because it is too costly to pay people to do so. There have already been a few viral stories of poorly-supervised coding agents deleting production databases or causing other types of damage. But we view these as "man bites dog" stories rather than an emerging norm. They go viral precisely because they represent such irresponsible and unusual behavior that they have shock value, and serve as regular reminders and learning moments helping the community guard itself against over-reliance on AI. As the aphorism goes, "if it's in the news, don't worry about it". Still, being able to detect whether there is an uptick in poorly-supervised use of AI for high-stakes tasks — across the economy, not just in software engineering — remains one of the most critical data gaps we have today.

By the way, the sandwich getting squished is a new trend and it is not uniquely due to AI. Over two decades ago, the Bureau of Labor Statistics started tracking programming separately from software engineering. Roughly speaking, programmers are responsible only for execution while software engineers manage a bigger part of the sandwich. Not only has programming been shrinking, it is also pays much less because it is seen as grunt work. AI merely accelerates this long-existing trend, further devaluing purely technical skills.





Software engineering versus programmer employment.
Chart by The Washington Post.

This pattern — where humans remain heavily involved at both ends of the decide-execute-deliver sandwich, even as AI increasingly automates the middle layer, seems to be broadly applicable to most knowledge work, though it is farthest along in software. After all, complex decision making and accountability are common to most fields. A lack of recognition of this phenomenon has led to many overconfident predictions about imminent job losses, such as among radiologists.

Vibe coding is not agentic engineering

One reason for confusion about the extent to which software engineering is changing is the sloppy use of the term “vibe coding” to refer to a wide spectrum of practices, the ends of which are conceptually distinct and more dissimilar than similar.

In true vibe coding the user simply tells the agent what to do, doesn’t supervise it when it’s running, doesn’t review the code — might not even

have the skills to do so — and doesn't evaluate the output, beyond perhaps noticing when things are visibly broken.

This is in contrast to how most software engineers are actually using agents — as a tool, with the human remaining in control and accountable for the output. Fortunately, the term agentic engineering is gaining currency as a descriptor of this practice.

As agentic engineering has become the norm, engineers are discovering that supervising coding agents is surprisingly time consuming. For example, Simon Willison, a prominent developer and chronicler of the AI transition, has noted how he is mentally exhausted by 11am from supervising agents. This is consistent with our experience as well.

More quantitative evidence comes from SWE-chat, a dataset of coding agent interactions from open-source developers who opted into a logging tool. The study found that only 44% of agent-produced code survives into user commits, that vibe-coded commits introduce vulnerabilities at nine times the human-only rate, and that the most common user intent is understanding existing code, not generating new code (19% vs 13%). The self-selected nature of the dataset means that we can't draw strong conclusions based on this study alone, but it does reinforce many other lines of evidence that vibe-coding and agentic engineering patterns are quite different.

	Vibe coding	Agentic engineering
Role of AI	Automation — AI does the work; the human delegates and steps back	Collaboration — AI amplifies human ability; the human stays in the loop, steering
Best suited for	Throwaways, prototypes, personal tools, demos, or one-off projects	Production software meant to be maintained, shipped, depended on
Who does it	Anyone — non-programmers, product managers, designers, hobbyists	Professional software engineers (requires high level of judgment)
Specification	Vague prompts; steer "by vibes," regenerate until it looks right	Precise specifications and success criteria; explicit constraints
Human	Low or none. No mental model of	Maintain a working model of

understanding	the architecture required	structure, data flow, invariants
Verification and debugging	Run it, eyeball it, "does it work?" (No need to read the code.) Just re-prompt, ask AI to fix, or work around bugs blindly.	Comprehensive tests, code review, guardrails, iteration and pushback. Diagnose and fix root causes.
Maintainability / refactoring	Not a concern. Version control not required.	Disciplined commits, reviewable history, branch hygiene
Security risks	Largely ignored	Explicitly handled through a combination of manual code review and automated guardrails.
Stakes and accountability	Low-stakes use cases; little or no accountability	High-stakes and mission-critical systems; human is accountable for failures
Skill erosion	Passive acceptance; risk of losing whatever skills the user has in the first place	Skill-preserving, skill-enhancing, or even reskilling-enabling, with the right workflows

Agentic engineering is not vibe coding

To re-iterate, these are not two distinct categories. They are two ends of a spectrum, and there is a blurry middle. Not every project is either a throwaway or mission-critical. Not every workflow fits precisely in the left column or the right column of the table. But the key implication for the jobs question remains solid — companies can't ship production software by hiring unqualified vibe coders instead of software engineers.

What does the future hold?

AI boosters might claim that mass layoffs are coming; they just haven't happened yet because human-level software engineering abilities are very recent (or haven't been achieved yet). But if the sandwich model is correct, these predictions won't come true. AI has *already* largely compressed the middle of the sandwich (and the compression actually started decades ago). So even making the execution layer instant and perfect will only be a small change from the status quo. The reasons why the other two layers have resisted AI is *not* because of capability limitations.

In fact, not only are software engineering jobs not going away due to AI, there

might even be an increase in demand for software engineers. When software (or anything else) gets cheaper to create due to technological productivity improvements, people will buy a lot more software (in econ jargon, software is highly "price elastic"). And as we have argued, AI doesn't replace software engineers (the "elasticity of substitution" is low), so the demand for more software results in a derived demand for more software engineers. A loosely related but flashier economics term, "Jevons' paradox", is often thrown around in the AI discourse to describe this concept.

Historically, this has been the pattern — programmer employment in the U.S. has grown from near-zero around 1950 to millions today. This is sharply different from occupations such as agriculture in which labor demand was famously decimated due to mechanization and automation. The difference is that the amount of calories people consume is relatively fixed — even a 25% increase led to the obesity epidemic — whereas the amount of software produced has grown a millionfold. Modern cars have something like a hundred million lines of code running on their various on-board computers.

If there is a ceiling to the demand for code, we are nowhere near it. Virtually all cognitive work benefits from software. As AI makes coding cheaper, people are creating all kinds of one-off utilities — whether for work or personal use — that it never made sense to create until now.

To be clear, while we think there will be a lot more software in the future, and likely more software engineers, this doesn't mean big tech companies will get even bigger. The majority of software engineers today already work in-house in non-software firms, and that share might grow in the future. Then there's the idea of "AI rollups", which refers to venture capital or private equity firms buying "Main street" businesses — dentistry practices, accounting firms, and whatnot — and rebuild them from the ground up to be "AI-native" by embedding software engineers or AI engineers into those businesses. Of course, it might end up being nothing more than hype. It's too early to tell.

Some people predict that demand for software engineering skills will fall because of democratization. They acknowledge that there will be more software produced than ever before, and also that more human time will be

spent producing software than ever before, but that this work will be done by people who are not software engineers. The idea is that AI will democratize software engineering to the extent that legal software, for instance, can be more easily created by those with training in law than in software engineering.

Maybe. But we'll bet against it. In our view, this falls into the same trap of conflating vibe coding with agentic engineering, and the execution layer with the the whole decide-execute-deliver sandwich. In fact, when we look at the history of programming, there have always been claims that we are at the threshold of democratization — old languages such as FORTRAN, COBOL, and SQL were all accompanied by such prominent hopes at the time of their introduction. It never happened. The barrier isn't actually learning the syntax. It's having enough skilled judgment to make good decisions while maintaining accountability.

Ultimately the distinction may be semantic. It seems clear that the amount of time people spend on getting computers to do new things will increase over time. This might take the form of building software, or managing complex workflows using agents, or something else. It will require a mix of software skills, AI skills, and domain expertise. Whether it is today's software engineers who will best adapt to fill these new roles remains to be seen.

That last point about the need for adaptation sets up the next essay in this series. The fact that aggregate labor demand in software is likely to remain strong doesn't mean that most *individual* workers won't be affected. We will argue that AI will create massive structural shifts in how software is produced, which will have big impacts on *which* software engineers stand to gain or lose — based on the types of firms they work in, their geography, their seniority, the pace at which they can adapt.

Further reading

Deena Mousa points out the superficiality of broad, economy-wide analyses of AI impacts based on metrics like "AI exposure", and instead calls for "careful, occupation-specific work". We hope that this series of essays will play a role in establishing a nuanced understanding of AI's transformation of software engineering. We've previous coauthored, with Justin Curl, a paper

analyzing AI in legal services that seriously engages with regulatory and other bottlenecks that make that occupation unique. We plan to do more occupation-specific deep dives in the future.

In a remarkable essay called *No Silver Bullet* 40 years ago, Fred Brooks distinguished between the "essential complexity" and "accidental complexity" of software. He argued that some of the complexity of software is accidental, arising from limitations of present technology such as the clunkiness of programming languages, and can be alleviated over time as tooling improves. But some of it is essential, because specifying the correct behavior of software is itself hard. He presents a forceful articulation of why the "decide" layer of the sandwich is thick and resists automation. Interestingly, hopes of boosting programmer productivity through AI were already prominent back then! Brooks argues that because AI or any other technology only reduces accidental complexity, it won't result in an order-of-magnitude productivity improvement. (Brooks is the author of *The Mythical Man Month*, an essay collection that is almost certainly the best known and most influential writing on software engineering of all time. *No Silver Bullet* later became part of the collection.)

We are grateful to Felix Chen for feedback on a draft.

---

1 The checkbox is actually labeled "technological innovation or automation". If checked, there is a second menu that to disclose the specific technology such as AI or robotics.

  The current WARN Act data have various limitations — it is New York only, and it is possible that companies are under-reporting AI as a reason for layoffs because of ambiguity or asymmetric risks from checking versus not checking the box (though we have no specific reason to think this). Stronger transparency requirements are in the works at both the federal and state levels; closing this data gap is urgent.

2 We are grateful to our colleague Mihir Kshirsagar for connecting us to the New York State Department of Labor and Elena Grovenger from the

department for a prompt response.

3 The paper uses the term coder, but it defines the term based on skills rather than roles, resulting in a broad sweep of jobs that is much broader than “coding”. Measurements based on industry, title, and skills cannot be easily compared to one another.

4 Interestingly, in a sub-study looking at mobile apps, the paper found that the usage of the resulting apps did not go up at all. This gets at one important difference between consumer and enterprise software. The former competes for a relatively fixed pool of attention; more apps published doesn’t mean more hours of app usage. But in enterprise software there is a lot of room for growth, as previously human processes can be software-mediated or automated.